Exploiting Native Hardware Heterogeneity: Custom GPSIMD Operators for Accelerating Emerging LLM Memory on AWS Trainium

Zifan He, Jason Cong
University of California, Los Angeles
zifanhe1202@g.ucla.edu, cong@cs.ucla.edu

Abstract

Emerging memory modules of large language models (LLMs), such as Engram in DeepSeek-V4.1-Flash, mix irregular integer work (a 64-bit n-gram hash and a sparse table lookup) with dense computation (a GEMM and a gate). A GPU-FPGA prototype that offloads the irregular steps accelerates Engram by up to 5.9×, which shows the promise of disaggregation on heterogeneous hardware. AWS Trainium provides this heterogeneity natively: each Trainium 3 NeuronCore pairs dense engines with an 8-core GPSIMD Engine, and all engines share the on-chip SBUF. However, the Neuron Kernel Interface (NKI) exposes the GPSIMD cores only through fixed built-in instructions. We build a toolchain[1] that compiles C++ custom operators for the GPSIMD cores and injects them into the compiled NKI kernel as `KERNEL_CALL` instructions that read and write the same SBUF tiles as the surrounding NKI code. On a hand-optimized NKI port of DeepSeek-V4.1-Flash, two GPSIMD operators speed up the n-gram hashing by 1.79× at decode, the table retrieval by up to 4.95×, and the whole Engram computation by up to 1.14×.

Disclaimer: Every Trainium conclusion and experiment in this work comes from the Neuron SDK 2.33 toolchain, run and measured on a single trn3.48xlarge instance. Everything is from open sourced code and packages. We installed the Neuron GPSIMD Toolchain v1.25, which is a legacy version used for trn1 instances. We then built the toolchain based on how the Neuron runtime loads and runs GPSIMD code, so treat all of the information as reference only. The information does not necessarily reflect how AWS implements things internally.

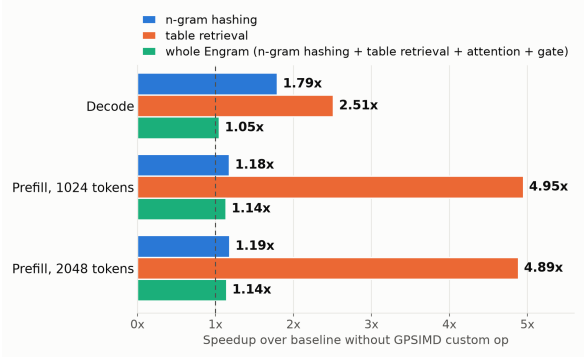


Figure 1: Speedup over the baseline without GPSIMD custom op (the hand-optimized NKI port), on one NeuronCore. Decode runs 10 sequences. Whole Engram counts both Engram layers.

[1] https://github.com/OswaldHe/neuron-gpsimd-tool

Contents

1 Introduction **3**

1.1 Heterogeneity of Computation Properties in LLMs 3

1.2 Example: Engram in DeepSeek-V4.1-Flash, and How We Accelerate It on a GPU-FPGA System 3

1.3 Native Hardware Heterogeneity of Trainium 4

2 Build and Use GPSIMD Custom Operators for Kernel Optimization **5**

2.1 Overall Development and Deployment Flow 5

2.2 Write Operators in C++ 6

2.3 Integrate the Custom Operator with NKI 7

2.4 Runtime: What Happens Under the Hood 9

3 Accelerating Engram in DeepSeek-V4.1-Flash with the GPSIMD Engine **10**

3.1 n-gram Hashing 10

3.2 Table Retrieval 11

3.3 Results and Analysis 13

4 What Other Operations Can GPSIMD Help With? **14**

5 Limitations **14**

1 Introduction

1.1 Heterogeneity of Computation Properties in LLMs

Modern LLMs depend on complex mechanisms to manage their memory: sparse attention selects key-value entries, retrieval-augmented generation (RAG) selects documents, and compressed memories select memory slots. Prior works published in ICML 2026 [13] showed that these mechanisms share a **memory processing pipeline** of four steps (Figure 2):

1. **Prepare Memory**: turn the memory into a form that is easy to search and use (for example, indexing vectors).
2. **Compute Relevancy**: score or identify the memory entries that matter for the current input.
3. **Retrieval**: select and fetch the relevant subset.
4. **Apply to Inference**: feed the retrieved memory into the rest of the model.

The key is that these steps have very different computational characteristics. Some steps are dense and regular computations, whereas others are irregular and data dependent. Therefore, a natural approach of acceleration is **fine-grain disaggregated** inference on **heterogeneous platforms**. He et al. [13] apply a GPU-FPGA heterogeneous system [11]: the dense steps stayed on the GPU and the irregular steps moved to the FPGA, for up to 2.2× faster inference and up to 4.7× lower energy. Such methodology is also transferable to more recent LLMs.

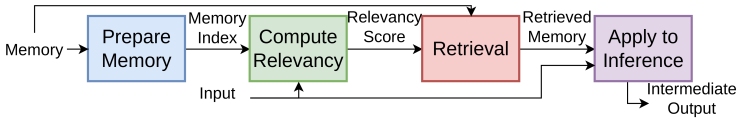


Figure 2: The four-step memory processing pipeline in LLMs, from [13].

1.2 Example: Engram in DeepSeek-V4.1-Flash, and How We Accelerate It on a GPU-FPGA System

DeepSeek-V4.1-Flash [8] is the recent 40-layer mixture-of-experts (MoE) reasoning model from DeepSeek. It combines sparse attention with MoE layers of 384 routed experts, 6 of which serve each token. Layers 1 and 14 also have an **Engram** module [7] that gives the model a large n-gram memory. For each token, Engram hashes the token and its preceding tokens into row ids, reads these rows from a large learned table, and adds a gated projection of the rows to the hidden state. Engram maps to the memory processing pipeline as follows:

- **Prepare Memory.** The table is trained offline. In DeepSeek-V4.1-Flash, each Engram layer has 384M rows of 256 bf16 values. At run time, each token's n-gram window is collected: the token's compressed id and the ids of the 3 preceding tokens.
- **Compute Relevancy.** An n-gram hash turns each window into row ids. For $n = 2, 3, 4$, the hash multiplies each token id by a 64-bit constant, combines the products with XOR, and reduces the result modulo one of 8 primes of about 16M. Each token gets $3 \times 8 = 24$ row ids per layer.
- **Retrieval.** The 24 rows of each token are read into an embedding `emb` of shape $[T, 24 \times 256]$.
- **Apply to Inference.** A GEMM projects `emb` to 4 keys and 1 value of 5120 values each. A gate adds the value to each of the 4 residual streams, scaled by the dot product of the stream's hidden state and its key.

We also tried to deploy Engram on our GPU-FPGA heterogeneous system to demonstrate the effectiveness of such abstraction [11]. Here we use the original demo of Engram in [9]:

- **FPGA: Compute Relevancy, Retrieval, and the key and value projections.** A hash unit computes the 16 row ids of one token per cycle. 16 lookup units read the rows from separate HBM channels. 8 projection units keep the weights in on-chip memory.
- **GPU: the gate, a short convolution, and the residual add.** These steps need the hidden state from the previous layer.

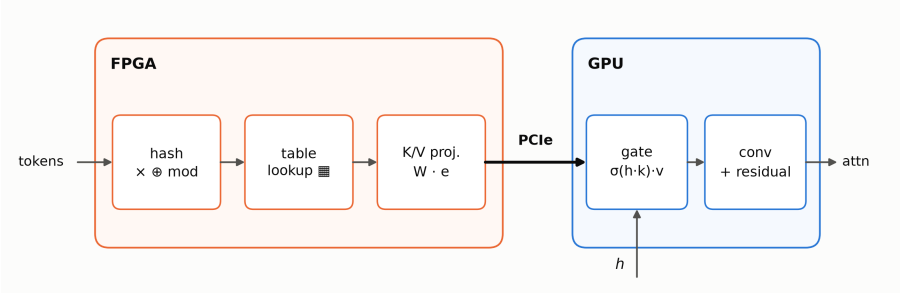


Figure 3: The GPU-FPGA split of Engram in HeteroLLM [11]. h is the hidden state.

The baseline is the reference code [9] on the GPU, which computes the hash on the host CPU. With 2 AMD U55C FPGAs and 1 AMD MI210 GPU, Engram is 5.2× faster at 8 tokens, 5.9× faster at 32 tokens and 1.15× faster at 1024 tokens (Figure 4). The gain is largest at few tokens, as in decode. There, most of the baseline time is a fixed cost per call: the hash on the host, the copy of the ids, and many small GPU operators. At 8 tokens, the baseline spends 0.98 ms on the hash alone, while the FPGA takes 0.39 ms for the hash, the lookup and the projections together.

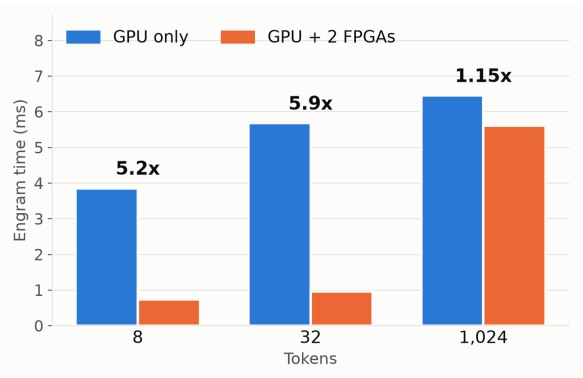


Figure 4: Engram on the GPU alone and on the GPU with 2 FPGA boards. Engram reference configuration [9].

The prototype shows that Engram gains from moving its irregular steps to flexible hardware. This technique is transferable to AWS Trainium devices: Trainium NeuronCore is natively heterogeneous.

1.3 Native Hardware Heterogeneity of Trainium

In this work, we conduct all the experiments on a Trainium 3 instance. A trn3.48xlarge instance exposes 64 NeuronCores (each made of 2 physical cores). A Trainium 3 NeuronCore is a heterogeneous processor (Figure 5). The **Tensor Engine** is a 128×128 systolic array for matrix multiplication. The **Vector Engine** and the **Scalar Engine** apply element-wise operations, reductions and activation functions to tiles of up to 128 partitions. The **Sync Engine** issues queued DMA transfers. All engines read and write the on-chip **SBUF** (state buffer), and the Tensor Engine accumulates into **PSUM**.

The fifth engine, the **GPSIMD Engine**, is different from the other four. The GPSIMD Engine is a group of 8 general-purpose SIMD cores. Each core has its own local data RAM, an integrated DMA channel (iDMA), and a compute unit that support 64-bit wide arithmetics. The cores run firmware that implements the GPSIMD Engine's instructions in software.

Each engine fits a different kind of computation, so each step of Engram has a natural place (Table 1).

The n-gram hashing and the table retrieval go to the GPSIMD cores because both are irregular integer work. The hash needs 64-bit products and a 64-bit modulo: the cores compute the products, while the other engines compute exact integer arithmetic only on 32-bit values. In the table retrieval, only about 1.5% of a token's lookups hit the table shard of a rank. GPSIMD engine can drive the table

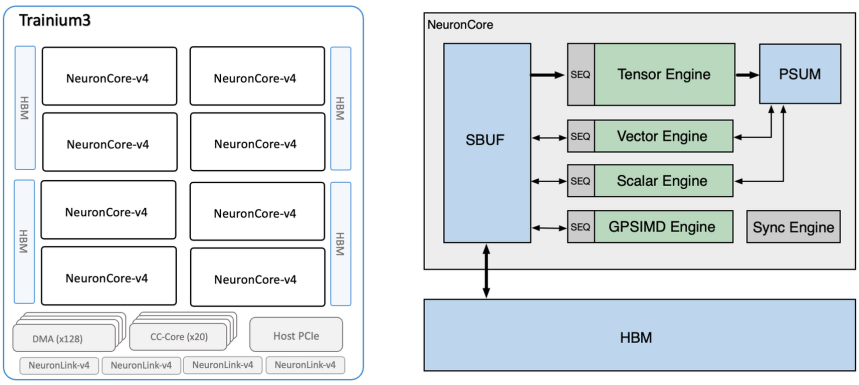


Figure 5: Trainium 3 device diagram and the architecture of a NeuronCore, from the Neuron SDK documentation [5, 6].

Table 1: The engines of a Trainium 3 NeuronCore and the Engram computations that suit them.

Engine	**Specialized computations**	**Which computation in Engram?**
Tensor Engine	dense matrix multiplication	the dense GEMM that projects the retrieved embeddings to the keys and the value
Vector and Scalar Engines	element-wise floating-point math, reductions, activations	the gating network
DMA/Sync engines	bulk copies between HBM and SBUF	reading the table rows
GPSIMD Engine	integer math, per-element branches, variable-length output	the n-gram hashing, and finding which lookups of the table retrieval hit this rank's table shard

lookups on these entries in the table to issue DMA. The dense GEMM and the gate stay on the Tensor, Vector and Scalar Engines, as GPSIMD engine has much lower peak performance in dense linear algebra than other engines. In general, a step suits the GPSIMD cores when it is integer-heavy, data-dependent, and small in data compared with the dense work around it.

Although this is a promising plan, a major blocker for most developer on Trainium is that there is no official support to connect custom GPSIMD operations with the kernel language in Trainium to optimize the whole kernel. In the next section, we will illustrate how to integrate custom GPSIMD operator with Neuron Kernel Interface (NKI) for a coherent optimization.

2 Build and Use GPSIMD Custom Operators for Kernel Optimization

The **Neuron Kernel Interface (NKI)** [2] is the Python kernel language for Trainium to speedup computations. Currently, NKI exposes the GPSIMD Engine only through built-in instructions on Trainium 3: tensor operations with `engine=gpsimd`, `iota`, `topk`, and the descriptor generation of indirect DMAs ("software DGE"). In the legacy SDK, there used to be a GPSIMD toolchain to create custom operator and integrate into `torch_xla` compiled kernel. However, this means if the user applied GPSIMD custom operator, the kernel can no longer be optimized with NKI. Furthermore, this package is deprecated on Trainium 3: when using the C++ custom-operator path (`AwsNeuronCustomOp`) that works on Trainium 1, the compiler rejects that path for Trainium 3 targets (error `NCC_EVRF030`). Therefore, for Trainium, there is no standard way to integrating GPSIMD custom operator nowadays. To make the whole stack works for Trainium, we reverse engineered and build a custom toolchain to inject the GPSIMD operator in Neuron Executable File Format (NEFF).

2.1 Overall Development and Deployment Flow

The Neuron compiler turns an NKI kernel into a NEFF, which holds one instruction stream per engine [12]. Our toolchain does not change the compiler. The NKI kernel is compiled as usual, and

the toolchain then edits the compiled NEFF to place each custom operator in the GPSIMD Engine's instruction stream. The rest of the NEFF, including the compiler's schedule of the other engines, stays as the compiler produced it. Figure 6 shows the flow in five steps:

1. **Write** the operator in C++ using the `gp::` namespace with the `gpsimd/gpsimd.hpp` header.
2. **Mark** the call site in the NKI kernel with `gp.call(...)`, and compile the kernel with the NKI compiler as usual.
3. **Build** the operator library: the toolchain reads the SBUF location of each tile from the compiled NEFF and compiles the C++ operator for the GPSIMD cores so that it can access the corresponding SBUF address.
4. **Inject** the library into the NEFF, and replace the marked instructions with one `KERNEL_CALL` instruction.
5. **Load and run** the NEFF with the Neuron runtime, which will load the shared object for the operator when hit the `KERNEL_CALL` and run on GPSIMD.

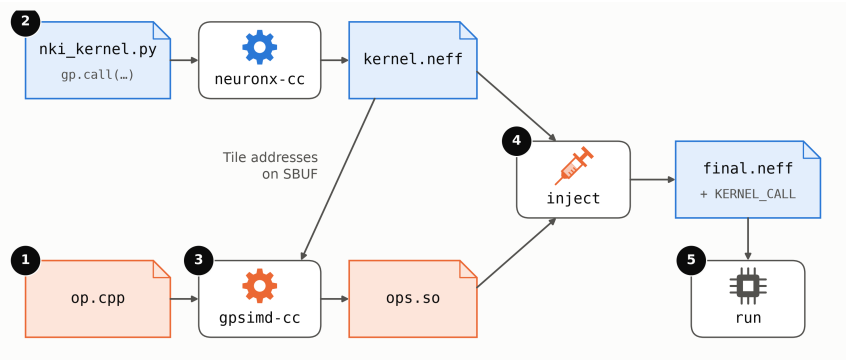


Figure 6: Overall development and deployment flow of building and using GPSIMD custom operator and connect with optimized NKI kernel.

`gpsimd-build` in our toolchain will handle all the compilation and injection process to produce the final NEFF file. The user only need to provide the C++ file for the GPSIMD operator and the NKI kernel python file with `gp.call()` that call the function in GPSIMD operator.

2.2 Write Operators in C++

A GPSIMD operator is a C++ function that can express the execution of all 8 cores (SPMD similar to CUDA kernel or NKI SPMD for LNCs). The runtime library gives each core typed views of the SBUF tiles passed by `gp.call`, its share of the partitions, and asynchronous iDMA copies between SBUF, HBM and the core's local buffer. The example below adds two int32 tiles:

```
#include <gpsimd/gpsimd.hpp>
GPSIMD_OP(vadd_i32) {                          // extern "C" void vadd_i32(gp_call *), runs on all 8 cores
    gp::Call c(call);
    auto a = c.in<int32_t>(0), b = c.in<int32_t>(1);      // SBUF tiles passed by gp.call
    auto o = c.out<int32_t>(0);
    const uint32_t n = o.elems();
    int32_t la[4096], lb[4096];                           // staging buffers in local data RAM
    for (gp::Range blk : c.blocks(o.partitions(), 4096 / n)) {   // this core's partitions
        c.fetch(a, blk.lo, blk.size(), la);                       // iDMA SBUF -> local RAM
        c.dma_wait(c.fetch(b, blk.lo, blk.size(), lb));
        for (uint32_t k = 0; k < blk.size() * n; ++k) la[k] += lb[k];
        c.dma_wait(c.flush(o, blk.lo, blk.size(), la));           // iDMA local RAM -> SBUF
    }
}
```

The operator can access the core ID with `c.core()` (0 to 7) and the number of cores with `c.ncores()` (8). Most operators use the index to access partition of data through `c.split(n)`, which gives core k the k-th of 8 equal blocks of $[0, n)$, or `c.blocks(n, rows)`, which walks that block in chunks of at most `rows` partitions. In the example above, each core takes 16 of the 128 partitions. The same split written by hand, with direct element access to SBUF instead of iDMA staging:

```
const uint32_t k = c.core(), K = c.ncores();            // this core (0..7) and the number of cores (8)
const uint32_t P = o.partitions(), per = (P + K - 1) / K; // 16 partitions per core for P = 128
for (uint32_t p = k * per; p < (k + 1) * per && p < P; ++p)
    for (uint32_t i = 0; i < n; ++i)
        o.store(p, i, a.load(p, i) + b.load(p, i));      // each SBUF load takes about 296 cycles
if (k == 0) gp_printf("done: %u partitions\n", P);       // work for one core only
```

In practice, an operator rarely accesses SBUF element by element. As in the first example, it copies a block of its partitions into local RAM with `c.fetch`, computes there, and writes the result back with `c.flush`, because local RAM is much faster to access than SBUF. Table 2 lists the functions of the runtime library.

Table 2: Functions of the operator runtime library.

Group	Functions	Purpose
Cores and work split	`c.core()`, `c.ncores()`, `c.split(n)`, `c.partitions(t)`, `c.blocks(n, rows)`	this core's index and its share of the work
Tiles	`c.in<T>(k)`, `c.out<T>(k)`	views of the SBUF tiles passed by `gp.call`; `T` is `int8_t` to `uint32_t`, `gp::f32`, `gp::bf16` or `gp::f16`
Element access	`t.load(p, i)`, `t.store(p, i, v)`, `t.row(p)`, `t.partitions()`, `t.elems()`, `t.contiguous()`, `t.span()`, `t.offset(i)`	element `i` of partition `p`, row-major over the free dimensions
iDMA copies	`c.fetch(t, p, n, local)`, `c.flush(t, p, n, local)`, `c.dma(dst, src, bytes)`, `c.dma_2d(dst, src, row_bytes, nrows, src_pitch, dst_pitch)`, `c.dma_wait(ticket)`, `c.dma_wait_all()`	asynchronous copies between local RAM, SBUF and HBM, completed in order
HBM	`gp::hbm_ptr<T>(params, k)`, `gp::hbm_mapped(params)`	direct access to HBM tensors that the runner maps
Cooperation across cores	`c.barrier()`, `c.allreduce_add(v)`, `c.allreduce_max(v)`, `c.allreduce_min(v)`, `c.exscan_add(v, &total)`, `c.lock()`, `c.unlock()`, `c.atomic_add(word, v)`, `c.shared(&n)`	synchronization and data exchange through the call's SBUF scratch tile; all cores must call them in the same order
Integer	C++ integer arithmetic, including 64-bit `* / %`; `memcpy`, `memmove`, `memset`, `memcmp`; inline assembly of base-ISA instructions such as `muluh`	64-bit operations run as runtime helpers
Floating point	`gp::f32` (`+ - * /`, comparisons, `to_i32()`), `gp::fma`, `sqrt`, `min`, `max`, `abs`, `floor`, `ceil`, `trunc`, `rint`, `round`, `ldexp`; `gp::bf16`, `gp::f16`; `GP_F32(x)`	IEEE float32 in software, bit-identical to numpy
Math	`gp::math::exp`, `log`, `tanh`, `sigmoid`, `sin`, `cos`, `gelu`, `rsqrt`, `isnan`, `isinf`, `copysign`, `frexp`	float32 functions

Next, we show how an NKI kernel calls this function.

2.3 Integrate the Custom Operator with NKI

In the NKI kernel, `gp.call` takes the operator name and the SBUF tiles that the operator reads and writes:

```
import gpsimd_nki as gp

@nki.jit
def kernel(a, b):
    out = nl.ndarray((128, 512), dtype=nl.int32, buffer=nl.shared_hbm)
    ta, tb, tc = (nl.ndarray((128, 512), dtype=nl.int32, buffer=nl.sbuf) for _ in range(3))
    nisa.dma_copy(dst=ta, src=a); nisa.dma_copy(dst=tb, src=b)
    gp.call("vadd_i32", ins=[ta, tb], outs=[tc])        # runs on the 8 GPSIMD cores
    nisa.dma_copy(dst=out, src=tc)
    return out
```

Because the NKI compiler does not know custom operators, `gp.call` cannot emit the call itself. Instead, it emits **marker instructions**: ordinary GPSIMD Engine instructions (`TENSOR_SCALAR_ARITH_OP` and `MEMSET`) whose immediate operand carries a tag ($2^{20} + 4096 \cdot \text{site} + 512 \cdot \text{role} + 32 \cdot \text{argument} + \text{chunk}$) so that the injector will not confuse with other instructions.

The tag tells the toolchain which call site, role and tile a marker belongs to. The marker indicates what the GPSIMD operator will access and modify (Table 3).

Table 3: Marker instructions emitted by `gp.call`.

Marker	What it does	What the compiler then does for the operator
touch	updates each input tile in place	allocates the input tiles in SBUF and keeps them alive until the call
join	fully writes each output tile	allocates the output tiles
chain	links the markers through a small token tile	orders every producer of an input before the call and every consumer of an output after it
fill	writes a scratch tile	reserves SBUF scratch for cooperation across cores

Next, the toolchain analyzes the compiled NEFF and injects the operator into it. A NEFF is a container with a 1024-byte header followed by a gzip-compressed tar archive:

```
kernel.neff
|-- [header, 1024 B]          payload size, MD5 of the payload, and other metadata
`-- [payload, tar.gz]
    |-- neff.json             top-level graph: input tensors -> kernel -> output tensors
    |-- kelf-0.json           list of subgraphs and the target ("mariana" = Trainium 3)
    |-- info.json             compiler and NKI versions
    `-- sg00/                 program of one physical core
        |-- def.json          memory variables, DMA queues and their semaphores, engine-to-stream map
        |-- tensor_map.json   data type, shape and layout of each input and output tensor
        |-- PE0.bin           Tensor Engine instruction stream
        |-- Activation0.bin   Scalar Engine instruction stream
        |-- DVE0.bin          Vector Engine instruction stream
        |-- Pool0.bin         GPSIMD Engine instruction stream
        |-- SP0.bin           Sync Engine instruction stream
        |-- {PE0,Activation0,DVE0,Pool0,SP0}.json            description of each stream
        |-- act_info.json, <set>.json, <set>_{bkt,ctrl}.bin  activation tables of the Scalar Engine
        |-- default_*_table.bin                              Vector Engine microcode tables
        `-- debug_info_*.dbg                                 debug information
```

`neff.json` describes the NEFF as a graph from the input tensors through the kernel to the output tensors, `kelf-0.json` lists the subgraphs, and `info.json` records the compiler versions. Each instruction stream (`<engine>0.bin`) is an array of 64-byte instructions. Figure 7 shows the layout of the two instructions that the toolchain reads and writes, with the values of the n-gram hashing kernel. The first 12 bytes have the same layout for every opcode: a header that starts with the opcode, and the **events block**, which holds one semaphore wait and one semaphore update. The rest of the slot holds the operands. A tensor operand is an address or an access pattern in NKI's `.ap()`.

The engines run independently and synchronize only through **semaphores**. When an instruction finishes, it can add to a semaphore: GPSIMD Engine instructions use S[3], and DMA transfers use the semaphores of their queue. A wait such as `S[3] >= 28` in Figure 7 counts the increments since the start of the kernel, so the schedule of the whole kernel is encoded in these counts. Adding or removing an instruction that increments a semaphore shifts every later wait on that semaphore, which is why injection must modify the events bytes in the instructions.

`gpsimd-neff analyze` reads the compiled `kernel.neff` and finds the tagged markers in the GPSIMD Engine's instruction stream (`Pool0.bin`). The marker operands give the SBUF location of every tile: the first partition, the number of partitions, the byte offset inside a partition, the data type, and the free-axis access pattern. In the join marker of Figure 7, which writes the output tile of the n-gram hashing call site, the second immediate holds the tag $2^{20} + 512 \cdot 5$ (site 0, role 5: join). The destination address is `0x4600`, and the operand covers 128 partitions with 48 int32 values each. Correspondingly, we hardcode the SBUF addresses in the custom operator.

`gpsimd-neff inject` adds two files and one key to the subgraph folder, rewrites the instruction streams, and repacks the archive with a new payload size and MD5 in the header:

```
final.neff
`-- sg00/
    |-- Pool0.json              + "gpsimd_kernels_def": "gpsimd_kernels_def.json"
    |-- gpsimd_kernels_def.json + [{"name": "ngram_hash__s0", "lib": "/sg00/ops.so"}]
    |-- ops.so                  + the operator library (Xtensa ELF)
    `-- Pool0.bin                  markers replaced by one KERNEL_CALL per call site
```

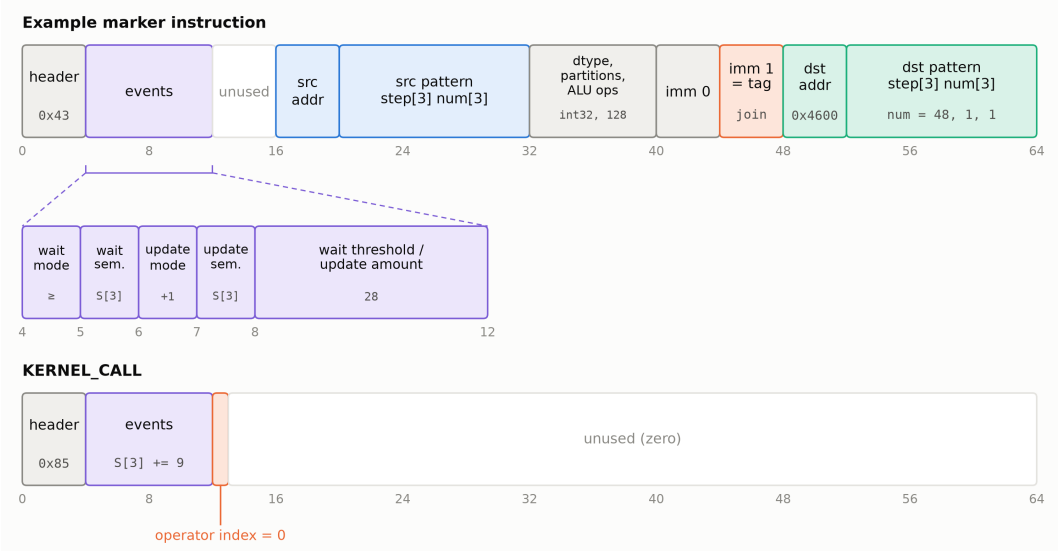


Figure 7: Layout of the 64-byte instructions that the toolchain reads and writes, with the values of the n-gram hashing call site. Top: the join marker emitted by `gp.call`, with its events block enlarged. Bottom: the `KERNEL_CALL` that replaces the markers.

In `Pool0.bin`, the markers of each call site become one `KERNEL_CALL` (opcode `0x85`) at the position of the site's first output marker. Byte 12 of the `KERNEL_CALL` is the index of the operator in `gpsimd_kernels_def.json`, and the other operand bytes are zero. The other markers become `EVENT_SEMAPHORE` instructions that keep the markers' waits, and the fill marker becomes a real `MEMSET` that clears the scratch tile. The bytes below come from the n-gram hashing kernel of the case study:

```
// Pool0.bin of kernel.neff (before), bytes 0-11
slot 67:  43 10 00 00 05 03 13 03 1c 00 00 00   TENSOR_SCALAR_ARITH_OP  wait S[3] >= 28; then S[3] += 1  (join marker)
slot 68:  d4 10 00 00 05 03 00 00 1d 00 00 00   PSEUDO_DMA_DIRECT2D     wait S[3] >= 29                 (reads the output)

// Pool0.bin of final.neff (after)
slot 68:  85 10 00 00 00 00 15 03 09 00 00 00   KERNEL_CALL            operator 0; then S[3] += 9 (SEM_ADD_IMM_COMPLETE)
slot 69:  d4 10 00 00 05 03 00 00 1e 00 00 00   PSEUDO_DMA_DIRECT2D     wait S[3] >= 30
```

The main challenge of injector is keeping track of the semaphore. The 9 markers of this call site each added 1 to S[3], so the replaced `KERNEL_CALL` instruction need to add 9 when the operator finishes. Before injection, a wait for a later GPSIMD Engine instruction also implied that the markers had finished. After the renumbering, every wait covers the same instruction as before and also the `KERNEL_CALL` of every earlier call site.

2.4 Runtime: What Happens Under the Hood

When the Neuron runtime loads the NEFF, it reads `gpsimd_kernels_def`, and the firmware copies `ops.so` from HBM into the local RAM of all 8 cores with the iDMA. At each `KERNEL_CALL`, the firmware calls the operator's entry function on all 8 cores. The entry function reads the core index from `PRID` and calls the operator.

The cores reach SBUF and HBM through address-translation windows. `gpsimd-run` (a C++ program on libnrt) programs one window over PCIe BAR after it loads the NEFF. During a call, the runtime borrows the iDMA channel from the firmware and restores the channel's descriptor pointer at the end of the call. After every execution, each core writes a receipt: its processor id, the number of calls, the call sites it ran, its cycle count and its status. The receipts prove that the operator ran on the 8 GPSIMD cores instead on the CPU host.

3 Accelerating Engram in DeepSeek-V4.1-Flash with the GPSIMD Engine

In this case study, we accelerate the Engram computation of the DeepSeek-V4.1-Flash port for Trainium 3 [10] with two GPSIMD operators. Figure 8 shows how one Engram layer is split between the GPSIMD cores and NKI on one NeuronCore. Compared with the GPU-FPGA split of Figure 3, the cut is finer: the GPSIMD cores take only the integer steps, which are the n-gram hashing and the hit compaction of the table retrieval. The gathers, the gate and the projection GEMM stay in NKI.

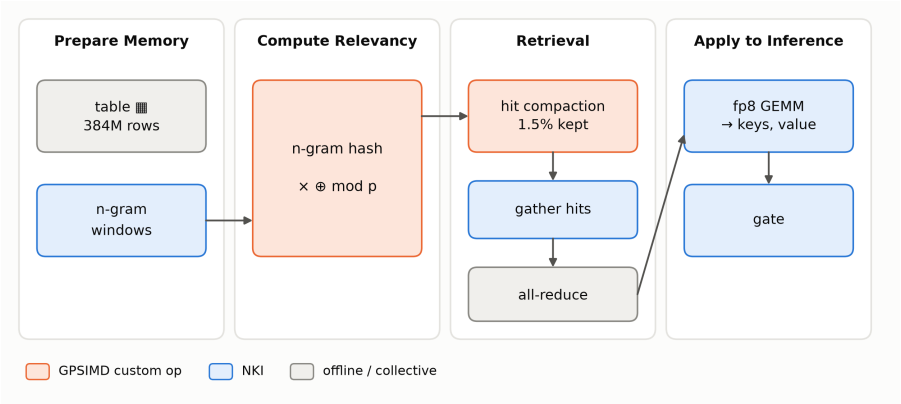


Figure 8: The Engram computation of one layer in the four-step memory processing pipeline of [13], split between GPSIMD custom operators and NKI.

The baseline is the hand-optimized NKI code of the port (branch `deepseek_v41_trn3`, commit `3b74533`). The port splits each Engram table by row over 64 ranks (6.0M rows, 3.1 GB per rank). Each rank reads only the rows it holds into `emb` and leaves the other rows zero, and an all-reduce over the 64 ranks then completes `emb`. For a fair comparison, each experiment copies the port's NKI helper modules without changes, runs the whole Engram computation on one NeuronCore, and changes only the part named in each section. The table shard has the deployed size (6,000,097 rows, 3.1 GB) with random values, and the row ids follow the hash distribution of the model. The collectives are not run. Latency is the median `total_exec_time` of 5 profiled executions in Neuron Explorer [3]. Every output of the GPSIMD version is checked against a numpy reference and against the NKI version: the n-gram ids equal an exact Python-integer reference, `emb` is exact, and the read-out output is bit-identical to the output of the port.

In the NKI port, two steps do work that the dense engines are not built for. The hash emulates 64-bit integer arithmetic with long chains of 32-bit and fp32 instructions, and the table retrieval fetches and masks every row, although only 1.5% of the lookups hit the rank. We replace each of the two with one GPSIMD operator.

3.1 n-gram Hashing

The port computes the n-gram ids of both Engram layers (48 ids per token) once per decode step, in its step prologue. A decode step of 10 sequences verifies 1 + 5 drafted tokens per sequence, so the hash runs on $T = 60$ rows.

The hash needs 64-bit products and a 64-bit modulo, but the NKI engines compute exact integer arithmetic only on 32-bit values and only in limited forms. The port therefore reads each product (token id $\times$ multiplier, as two 32-bit words) from a precomputed [99092, 16] table with an indirect DMA, and combines the words with XOR. It then reduces the 63-bit result with Horner's rule over 7-bit chunks, each step with an fp32 quotient estimate corrected by one prime. In the profile, the arithmetic part of the NKI hash is 230 instructions: 166 on the Vector Engine, 58 GPSIMD built-in instructions and 6 DMAs, mostly in one long dependent chain.

The GPSIMD version keeps the port's window gathers in NKI and replaces the arithmetic part with one operator, `ngram_hash`. Each core hashes its share of the rows in registers. `mulhi` is the `MULUH` instruction, which returns the high 32 bits of a 32-bit product, so a 64-bit product takes 3 multiplies. A Barrett reduction replaces the division by the prime:

```
// (token id x multiplier) mod 2^64 as two words: MULUH and MULL, then XOR into the rolling value
rh ^= mulhi(cid, ml) + cid * mh;
rl ^= cid * ml;
// v = rh * 2^32 + rl (v < 2^63) mod p, with mu = floor(2^64 / p) from the host:
// Barrett reduction, the quotient estimate is at most 3 too small
const uint64_t mid = (uint64_t)lo * muh + (uint64_t)hi * mul + mulhi(lo, mul);
const uint32_t q = hi * muh + (uint32_t)(mid >> 32);
uint32_t r = lo - q * p;
if (r >= p) r -= p;
if (r >= p) r -= p;
if (r >= p) r -= p;
```

Figure 9 shows where the time goes. Both versions spend the first 34 µs on the same NKI window gathers. After the gathers, the NKI version runs a long chain of small instructions on the Vector Engine and the GPSIMD built-ins for 62 µs, while the GPSIMD version runs one 19.5 µs `KERNEL_CALL`. Without counting semaphore waits, the GPSIMD version issues 107 instructions instead of 324. At decode, the hash drops from 86.95 µs to 48.45 µs (1.79×).

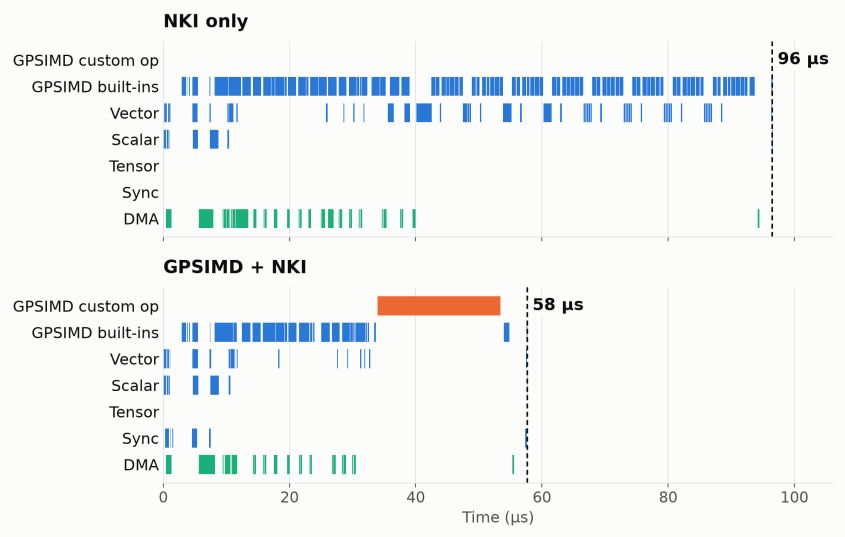


Figure 9: One profiled execution of the hashing at decode (10 sequences, $T = 60$). The window gathers (first 34 µs) are the same NKI code in both versions. The NKI arithmetic takes the next 62 µs; the GPSIMD operator takes 19.5 µs.

The operator was not faster from the start. The first version processed one row at a time and reduced each value with Horner's rule over 8-bit chunks, with one 32-bit Barrett step per chunk (5 steps per value). At 93.8 µs, it was slower than NKI (0.93×). Processing column by column keeps the constants of each prime in registers, and skipping the padding rows of the 128-row tile brought the time to 62.2 µs. The final 64-bit Barrett reduction above needs 6 multiplies and at most 3 corrections per value, for 48.45 µs.

The gain is smaller on full prefill tiles: 531.1 µs to 449.7 µs (1.18×) at $T = 1024$, and 1055.6 µs to 890.6 µs (1.19×) at $T = 2048$. On 128-row tiles, each NKI instruction does useful work on all 128 partitions, while at decode the same chain runs on 60 rows. (The port hashes prefill chunks on the host, so the prefill numbers show what a device-side hash would cost.)

3.2 Table Retrieval

Each hash column j covers its own range of about 16M global rows, and rank r holds 6.0M consecutive rows. A rank's shard therefore overlaps at most 2 of the 24 columns of a layer, and only about 0.375 of a token's 24 lookups (1.5%) hit the rank. In our prefill chunk of $T = 1024$ tokens, only hash columns 7 and 8 of a layer overlap the rows of rank 21, and the rank holds 372 of the 24,576 rows.

The port's `engram_gather` cannot know in advance which ids hit, because NKI needs static shapes. The port therefore computes a mask for every id, issues one indirect DMA of 128 rows per 128-token tile and hash column (192 DMAs at $T = 1024$), and multiplies the result by the mask, so it fetches all 24,576 rows to keep 372 of them. Every index of an indirect DMA costs about 5.5 ns of descriptor generation on the GPSIMD Engine, even when the index is marked out of bounds and skipped: in a separate test, 64 gathers of 640 rows took 224 µs whether 100%, 25% or 0% of the indices were valid. The 98.5% of indices that miss therefore cost as much as the hits.

The GPSIMD version lists the hits first and fetches only those rows. One operator, `engram_hits`, replaces the gather loop, and one short NKI loop fetches the hits:

```
// GPSIMD: each core scans its 16 partitions (all tiles, all 24 columns) after one iDMA fetch
for (uint32_t i = 0; i < NT; ++i)
    for (uint32_t j = 0; j < NCOL; ++j) {
        const uint32_t v = (uint32_t)r[i * NCOL + j];       // shard-local row id
        if (v < ROWW) { src[n] = v; dst[n] = (i * 128u + p) * NCOL + j; ++n; }
    }
const uint32_t off = c.exscan_add(n, &total);               // place each core's list
// one iDMA per list; core 0 pads to a multiple of 128 and writes the number of 128-hit chunks
```

```
# NKI: zero-fill emb, then one gather and one scatter of 128 rows per chunk of hits
gp.call("engram_hits", ins=[ids_sb], outs=[hits_sb, cnt_sb])
reg = nisa.register_alloc(); nisa.register_load(reg, cnt_sb)
nl.fori_loop(0, reg, chunk, step=1)     # chunk(b): load 128 (table row, emb row) pairs, gather, scatter
```

At $T = 1024$, the operator takes 36 µs, and NKI then needs 6 indirect DMAs (3 gathers and 3 scatters) instead of 192 gathers. The retrieval stage drops from 506.9 µs to 102.4 µs (4.95×), and from 980.5 µs to 200.7 µs (4.89×) at $T = 2048$. Figure 10 shows the two profiles. In the NKI version, the GPSIMD Engine is busy with descriptor generation and the Vector Engine with mask arithmetic for the whole stage. In the GPSIMD version, one 36 µs operator call and 6 DMAs replace this work.

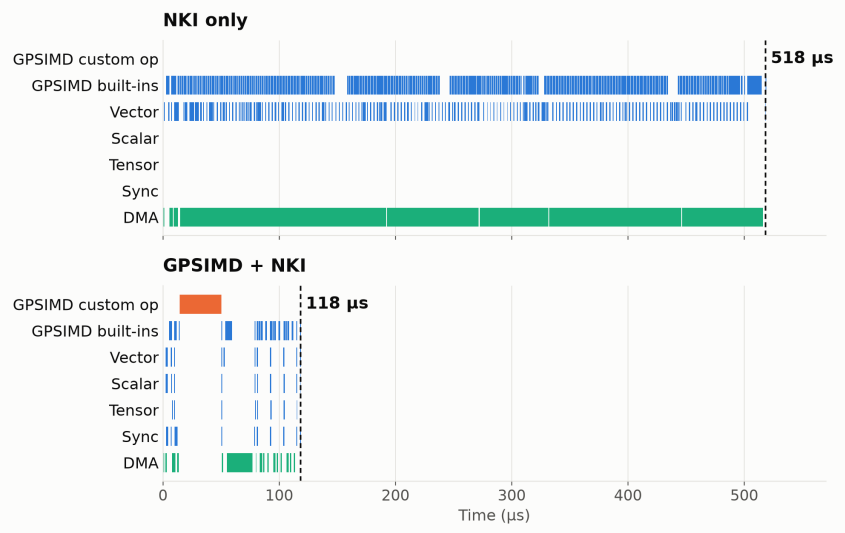


Figure 10: One profiled execution of the retrieval stage ($T = 1024$). Top: 192 indirect DMAs and their masks. Bottom: the `engram_hits` operator (orange, 36 µs), then 3 gathers and 3 scatters of hit rows.

At decode ($T = 60$), the run-time loop costs more than it saves. At every block boundary of `nl.fori_loop`, every engine drains and synchronizes. This stopped the weight stream of the following GEMM from overlapping with the gather, which slows down the kernel. A token has at most 2 hits, so a second operator, `engram_hits_t`, writes the hits directly in the layout of a fixed number of indirect DMAs (2 gathers and 2 scatters per 128-token tile), with the unused entries marked out of bounds. With this static schedule, the retrieval stage takes 24.7 µs instead of 62.1 µs (2.51×), and the read-out keeps its overlap.

3.3 Results and Analysis

Table 4 lists all results. The read-out is one Engram layer: table retrieval, fp8 GEMM and gate. Whole Engram is the n-gram hashing (one call computes the ids of both Engram layers) plus the read-out of both layers, as in Figure 1. In every row, the fp8 GEMM, the gate and all data movement stay in the port's NKI code; the two operators only compute the ids and the list of hits.

Table 4: Latency of the Engram computations on one NeuronCore, without (NKI) and with GPSIMD custom operators (GPSIMD + NKI).

Engram computation	Config	NKI only	GPSIMD + NKI	Speedup
n-gram hashing	decode, 10 sequences ($T = 60$)	86.95 µs	48.45 µs	**1.79**×
	prefill chunk, $T = 1024$	531.1 µs	449.7 µs	1.18×
	prefill chunk, $T = 2048$	1055.6 µs	890.6 µs	1.19×
retrieval stage	decode ($T = 60$)	62.1 µs	24.7 µs	**2.51**×
	prefill chunk, $T = 1024$	506.9 µs	102.4 µs	**4.95**×
	prefill chunk, $T = 2048$	980.5 µs	200.7 µs	**4.89**×
whole read-out	decode ($T = 60$)	697.1 µs	681.0 µs	1.02×
	prefill chunk, $T = 1024$	3362.9 µs	2968.9 µs	**1.13**×
	prefill chunk, $T = 2048$	6766.4 µs	5926.4 µs	**1.14**×
whole Engram (hashing + 2 × read-out)	decode ($T = 60$)	1481.2 µs	1410.5 µs	**1.05**×
	prefill chunk, $T = 1024$	7256.9 µs	6387.5 µs	**1.14**×
	prefill chunk, $T = 2048$	14588.4 µs	12743.4 µs	**1.14**×

Because the projection and the gate are unchanged, the speedup of the whole read-out depends on how much of the read-out the retrieval stage takes. Figure 11 shows the whole read-out at $T = 1024$. The GEMM on the Tensor Engine reads `emb`, so in the NKI version, most of the GEMM runs after the last gathered rows arrive at about 506 µs. In the GPSIMD version, the GEMM starts at 124 µs. The GEMM ends at 1,717 µs in the NKI version and at 1,318 µs in the GPSIMD version, about 400 µs earlier. The gate after the GEMM takes the same 1.66 ms on the Vector Engine in both versions.

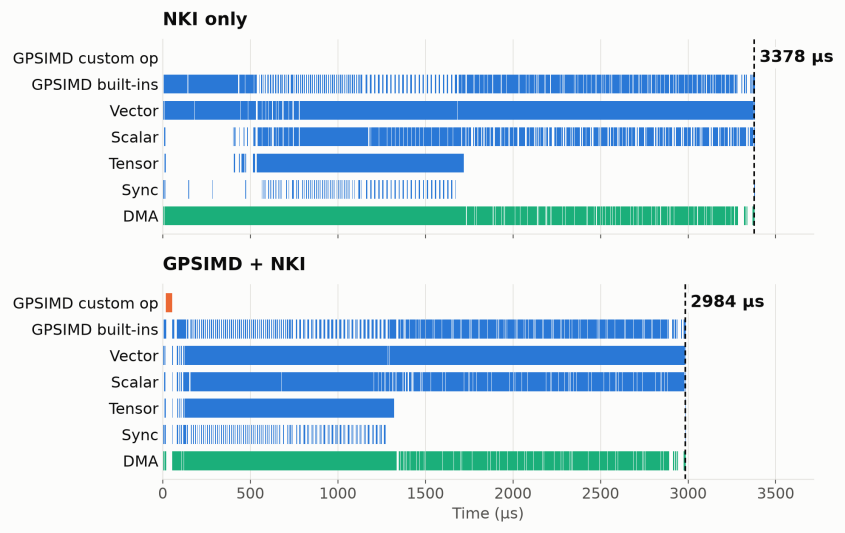


Figure 11: One profiled execution of the whole Engram read-out ($T = 1024$). The GPSIMD operator (orange, 36 µs) lets the GEMM start about 400 µs earlier; the rest of the read-out is unchanged.

4 What Other Operations Can GPSIMD Help With?

Nothing in this method is specific to Engram. Any step that is integer-heavy, data-dependent and small in data compared with the dense work around it is a candidate (see Section 1.3). Before Engram, we used the same toolchain on kernels of the NKI Library [4]. Against the library's NKI implementations, we measured the speedups in Table 5.

Table 5: Speedups of GPSIMD operators over the NKI Library implementations.

Kernel	**Speedup over the NKI Library**
argsort	6.6× to 40×
BM25 scoring	15× to 27×
scatter-add with duplicate indices	4.45×

5 Limitations

Currently, our toolchain supports only part of what the GPSIMD cores can do. The main gaps are listed below.

No compiler support for vector instructions. Each GPSIMD core has a 512-bit vector unit, and the unit is enabled while our operators run. However, the only Xtensa configuration in the Neuron SDK (`ncore2gp`) describes the Trainium 1 core. Trainium 3 encodes the vector instructions differently, so vector code generated by `xt-clang` hangs the core. The toolchain therefore compiles scalar base-ISA code only, and `gpsimd-cc` rejects any library that contains vector instructions.

No hardware floating point. The floating-point instructions of `ncore2gp` also hang the core, and we have not found their Trainium 3 encodings. Floating point therefore runs in software (`gp::f32`): a float32 addition takes 117 cycles and a multiplication 97 cycles. Steps dominated by floating-point math still belong on the Vector and Scalar Engines.

Tile addresses fixed at build time. We did not find a way for the operator to read the `KERNEL_CALL` operands at run time, so the SBUF addresses of the tiles are compiled into `ops.so`. The library must be rebuilt whenever the kernel is recompiled, and it must fit in 64 KB of code and about 7 KB of static data.

An official support on GPSIMD custom kernel should resolve these issues.

Acknowledgments

This work is supported by the AWS Build on Trainium program at UCLA. We used AutoHelix [1] as the agentic loop harness to optimize the kernel, with special modifications to add constraints based on human experts. We would like to thank Ellen Hermansen from the Amazon Research Initiative team for providing AWS access and promotional credits, Amit Kachroo for providing technical insights in the Amazon Science Hub program, and James Hamilton for the continuous support on providing latest Trainium instances.

References

[1] AWS Labs. AutoHelix. https://github.com/awslabs/AutoHelix, 2026.

[2] AWS Neuron. About NKI. https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/get-started/about/index.html, 2026. AWS Neuron documentation.

[3] AWS Neuron. Neuron Explorer. https://awsdocs-neuron.readthedocs-hosted.com/en/latest/tools/neuron-explorer/index.html, 2026. AWS Neuron documentation.

[4] AWS Neuron. NKI Library. https://github.com/aws-neuron/nki-library, 2026.

[5] AWS Neuron. Trainium3 architecture. https://awsdocs-neuron.readthedocs-hosted.com/en/latest/general/arch/neuron-hardware/trainium3.html, 2026. AWS Neuron documentation.

[6] AWS Neuron. Trainium3 architecture guide for NKI. https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/guides/architecture/trainium3_arch.html, 2026. AWS Neuron documentation.

[7] DeepSeek-AI. Conditional memory via scalable lookup: A new axis of sparsity for large language models. *arXiv preprint arXiv:2601.07372*, 2026.

[8] DeepSeek-AI. DeepSeek-V4.1-Flash. https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash, 2026. Model card on Hugging Face.

[9] DeepSeek-AI. Engram reference implementation. https://github.com/deepseek-ai/Engram, 2026. `engram_demo_v1.py`.

[10] Zifan He. DeepSeek-V4.1-Flash port for Trainium 3 in vllm-neuron. https://github.com/OswaldHe/vllm-neuron/tree/deepseek_v41_trn3/contrib/models/deepseek_v41/src, 2026. Branch `deepseek_v41_trn3`.

[11] Zifan He. HeteroLLM: GPU-FPGA heterogeneous inference framework. https://github.com/OswaldHe/HeteroLLM, 2026. Engram implementation in `kernels/deepseek_engram`.

[12] Zifan He and Jason Cong. Accelerating AWS Trainium below kernel level: NEFF-based performance optimization. https://github.com/OswaldHe/neuron-neff-doc, 2026.

[13] Zifan He, Rui Ma, Yizhou Sun, and Jason Cong. Understand and accelerate memory processing pipeline for large language model inference. *arXiv preprint arXiv:2603.29002*, 2026.